

Distributed Systems HW3 – Section 2

Weather Data Analytics (HW2 Q8) using MapReduce and gRPC

Rajai Tarun Kanaiyalal (2025201034)

Yash Khator (2024101133)

1 Q1: Batch analytics using MapReduce

In HW2 we solved the weather analytics problem (Q8) with a sequential program and with MPI. Here we solve the same problem, with the same input and output, using MapReduce, and then compare it with our MPI program. The code and the full run instructions are in `q1_mapreduce/README.md`.

1.1 How we ran it

The assignment asks for Hadoop Streaming. Hadoop is not installed on the RCE cluster (there is no Hadoop module), and the course told us to use a Slurm script until it is fixed. So we wrote the mapper, combiner and reducer as normal C++ programs that read from stdin and write `key<TAB>value` lines to stdout. This is exactly what Hadoop Streaming expects, so the same programs will also run on Hadoop without any change. Our Slurm script does the same steps that Hadoop would do:

1. **Split**: cut the input file into P pieces (one per mapper).
2. **Map**: P tasks run in parallel, each doing `mapper | sort | combiner` on its own piece.
3. **Shuffle**: the master merges all the sorted combiner outputs into one sorted file.
4. **Reduce**: one reducer reads that file and prints the final answer.

1.2 Our MapReduce design

For every record, the mapper prints three lines, using these keys:

Key	Value	How two values are combined
G	count, sums, min/max, extreme count, hottest, coldest	add, min/max, hotter/colder
S<station>	count, temperature sum, rainfall sum	add
I<interval>	count (interval = timestamp / 60)	add
K	the K from the header line	keep

The main idea: **one record is treated as a small partial result with count 1**. So the mapper output, the combiner input and the combiner output all have the same format, and the combiner and reducer only need to know how to merge two lines with the same key. Since the merges are just additions, mins and maxes, the order does not matter. The answer is the same no matter how the input is split or how many times the combiner runs. We never merge averages, only sums and counts, and divide at the very end.

Top- K stations and the busiest interval can only be decided after *all* the counts are merged (the same bug we found in HW2 when each process kept its own top- K). That is why we use **one reducer**: it sees every station and every interval.

1.3 Important decisions

Table 1 lists the main decisions we took, why, and what effect we measured.

1.4 Correctness

- **Local tests (69, all pass)**: the 9 HW2 test cases with expected outputs (ties, extreme values, $K > S$, $N = 0$) give the exact expected output. All 13 HW2 inputs plus generated 100K and 1M inputs, each split into 1, 2, 4 and 8 pieces, give the same output as the HW2 sequential program.

Decision	Why	Effect
Use a combiner	The mapper prints 3 lines per record, and the same keys repeat a lot inside one piece.	10M records: 30M $\rightarrow$ 7M lines sent to the reducer; 52 s $\rightarrow$ 16 s.
Only one reducer	Top- K and busiest interval need all the data in one place.	Reducer takes only 0.5–3.3 s.
Mapper prints the numbers exactly as read	Printing them as doubles turned 24.49 into 24.489999999999998.	Mapper output 2.1 $\times$ smaller.
Read numbers with <code>strtod</code>	<code>stringstream</code> was very slow for millions of lines.	Combiner 2.9 $\times$ faster.
LC_ALL=C <code>sort</code>	The normal <code>sort</code> can ignore the tab and mix up keys like <code>\$1</code> and <code>\$10</code> .	Correct grouping.
Pipe <code>mapper sort combiner</code>	The full mapper output for 50M records is 7.5 GB, but our disk quota is 10 GB.	Nothing big is written to disk.
Same data, same node as MPI	A fair comparison with HW2.	Differences come from the model, not the setup.

Table 1: Main design decisions.

- **Cluster:** the same output as sequential on one node and on two nodes, and all 44 benchmark runs are correct.

One small rounding difference. From 10M records onwards, the last digits of `TOTAL_RAINFALL` differ, e.g. 249966638.539998 (sequential) vs 249966638.540026 (MapReduce). The true value is 249966638.54, so both are slightly off. A `double` only keeps about 15–16 digits, and adding 10 million numbers in a different order gives a slightly different rounding. Our HW2 MPI program shows the same difference against sequential when it uses 1, 2 or 4 workers. So our checker (`compare_outputs.py`) requires counts, ids and timestamps to match exactly, and allows decimal numbers to differ only by a relative 10^{-10} (or by one unit in the 6th decimal for a *rounding tie*, see Q2). Every run passed: 15 matched exactly and 25 differed only by this rounding.

1.5 Experimental setup

We used the HW2 dataset generator with seed 42, $K = 10$ and $S = 100$, and input sizes of 1M, 10M, 20M and 50M records (50M records is 2.1 GB). We ran MPI with 1, 2, 4 and 8 workers and MapReduce with 1, 2, 4 and 8 mappers. All runs of one benchmark were in the same Slurm job, on one node, on the same data. We measured the total time of each run (including reading the input), the time of each MapReduce stage, and the peak memory of every process.

1.6 Results

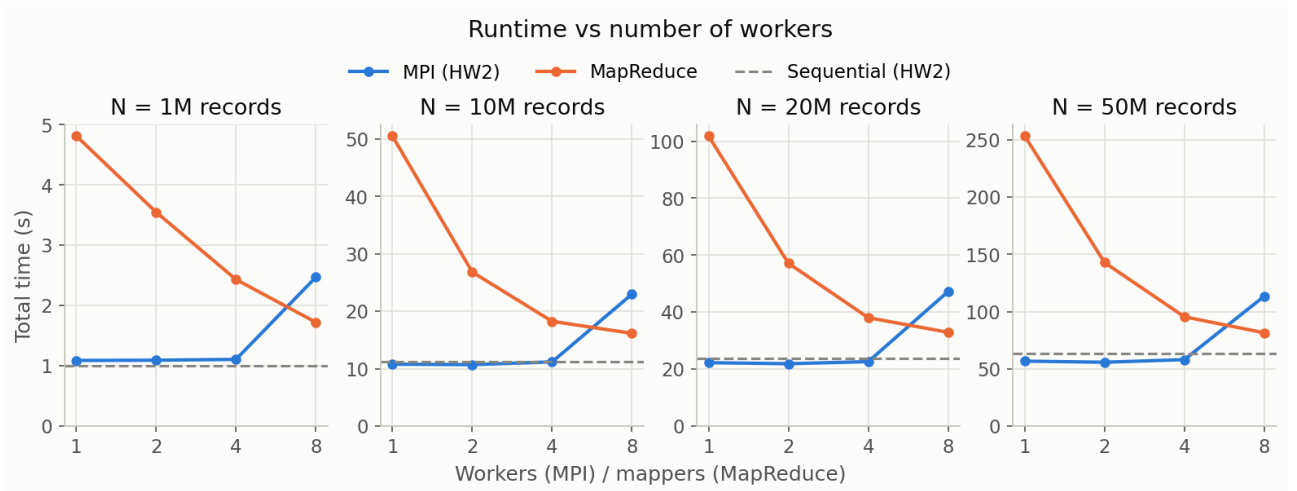


Figure 1: Total time vs number of workers/mappers. Dashed line: HW2 sequential.

N	Seq	MPI 1	MPI 2	MPI 4	MPI 8	MR 1	MR 2	MR 4	MR 8
1M	1.00	1.09	1.09	1.11	2.47	4.81	3.55	2.43	1.72
10M	11.22	10.78	10.70	11.19	22.98	50.51	26.83	18.24	16.16
20M	23.87	22.25	21.88	22.61	47.21	101.77	57.13	38.03	32.91
50M	63.65	56.74	55.78	58.06	113.48	253.44	142.90	95.46	81.50

Table 2: Total time in seconds (MR = MapReduce).

MPI is faster overall (Fig. 1, Table 2). At 50M records the best MPI run takes 55.8s and the best MapReduce run takes 81.5s. Both grow linearly with the input size. MapReduce does much more work per record. MPI reads each record once into a small binary struct and sends structs. MapReduce turns each record into 3 text lines (151 bytes, 3.6 times bigger than the input), sorts all these lines, and then reads the numbers back from text in the combiner. With only one mapper, MapReduce is 4 times slower than sequential (253s vs 64s).

MPI does not get faster with more workers. From our HW2 timing logs, at 50M records rank 0 spends about 46s of the 56s just *reading the input file alone*. The workers can only start after that, so adding more workers only speeds up the small compute part. With 8 workers the time even doubles, and the HW2 log shows that it is rank 0’s reading that becomes twice as slow. Our best guess is that the 8 waiting MPI processes keep the CPUs busy while they wait, which slows down rank 0.

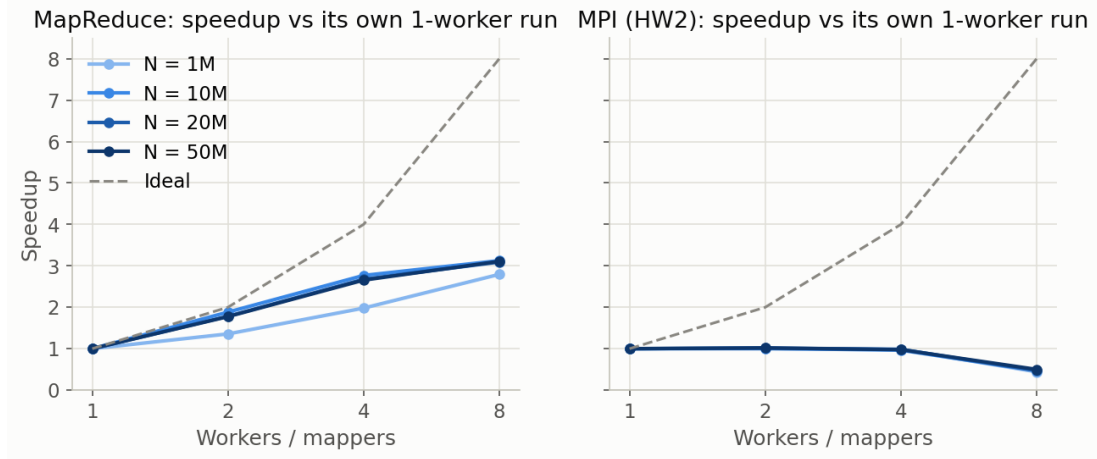
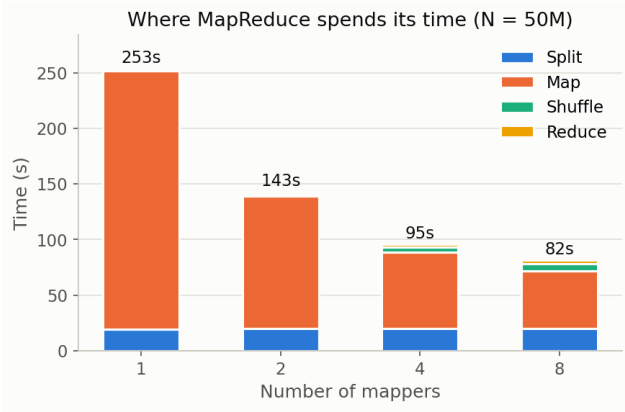


Figure 2: Speedup of each implementation compared with its own 1-worker run. Dashed line: ideal speedup.

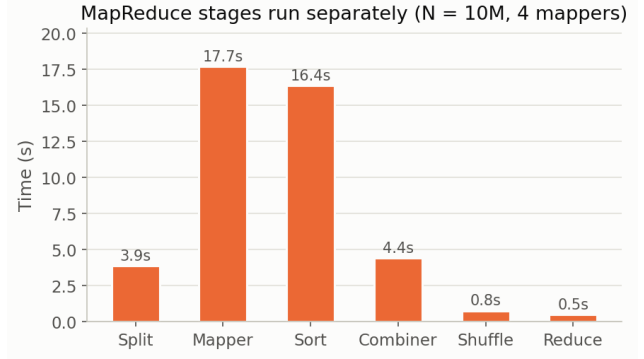
MapReduce gets faster with more mappers, but not 8 times faster (Fig. 2). In MapReduce each mapper reads and processes its own piece, so the slow part is done in parallel. At 50M records MapReduce is 3.1 times faster with 8 mappers than with 1. It does not reach 8 times because:

- **Splitting the input is done by one process** and always takes about 20s at 50M. That is 8% of the time with 1 mapper but 24% with 8 mappers. (In real Hadoop this step does not exist, because HDFS already stores the file in pieces.)
- **Shuffle and reduce get slower with more mappers** (2s with 1 mapper, 9.5s with 8), because more mappers send more lines to the reducer (explained below).
- **Each map task gets only one CPU core.** Slurm gives every task 1 core, so the mapper, `sort` and combiner of one task (three programs) share that core. All 8 tasks also share one node’s memory and disk.

Is it really running in parallel? To be sure, we ran `check_parallel.sh`: the map phase on 10M records in 8 pieces, where every task records its node, its CPU core and its start and end time, followed by the same 8 pieces run one after another. On one node, all 8 tasks started at the same moment (0.08s), ran on 8 different cores, and on average 7.8 of them were running at the same time. The parallel run took 12.5s versus 45.9s one after another (3.7× faster), with identical output. It is not 8× because each task has only one core for its three programs, while the one-after-another run could spread one pipeline over several cores. With `--nodes=4` the tasks ran on 4 different nodes (2 per node), so the multi-node run really is spread over machines.



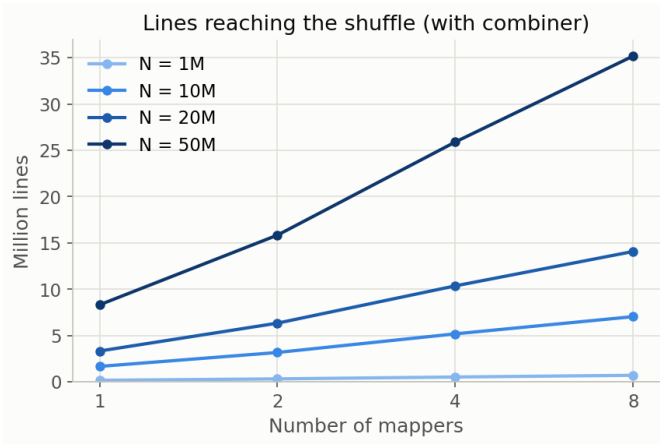
(a) Time of each stage, 50M records.



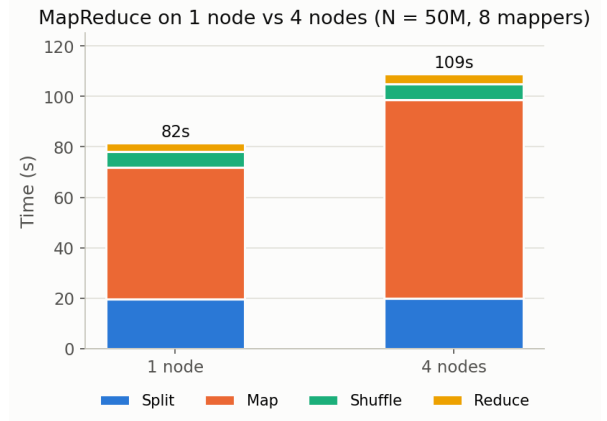
(b) Map phase split into parts, 10M records, 4 mappers.

Figure 3: Where MapReduce spends its time.

Where the time goes (Fig. 3a, 3b). The map phase (mapper + sort + combiner) takes almost all the time: 232 of 253s with 1 mapper at 50M. To see what is inside it, we also ran the three parts separately (10M records, 4 mappers): the mapper took 17.7s, the sort 16.4s and the combiner only 4.4s. So most of the time is spent on **converting records to text and sorting them**, not on the actual analytics. The mapper is slow because it splits every line into words and writes 1.5 GB of text for 10M records.



(a) Lines sent to the reducer.



(b) 1 node vs 4 nodes, 50M records, 8 mappers.

Figure 4: Data sent to the reducer, and running on more nodes.

More mappers send more data to the reducer (Fig. 4a). With 1 mapper at 50M, 8.3M lines reach the reducer; with 8 mappers, 35.2M lines. The reason: each mapper’s combiner can only merge the lines of its *own* piece. If the records of one interval are spread over 5 pieces, that interval is sent 5 times. Almost all of these lines are interval lines: there are about 8.3M intervals with only about 6 records each, so they get spread over many pieces. We checked this with a simple formula: an interval with on average 6 records, split randomly over P pieces, appears in $P(1 - e^{-6/P})$ pieces. This predicts 8.31M, 15.84M, 25.90M and 35.18M lines for 1, 2, 4 and 8 mappers, which matches our measurements.

The combiner makes a big difference (Fig. 5). Without the combiner, all 3 lines of every record go to the one reducer: 30M lines at 10M records. The shuffle then takes about 15s and the reducer 13s, instead of less than 1.3s each. With 8 mappers the whole run is 3.2 times faster with the combiner (16s vs 52s). Without it, adding mappers barely helps, because the single reducer becomes the bottleneck.

MapReduce uses much less memory (Fig. 6). This is where MapReduce clearly wins. In MPI, rank 0 reads *all* the records into memory before sending them out: 50M records $\times$ 56 bytes is about 2.8 GB, plus the hash map of all the intervals. So rank 0 used 3.4GB and each worker around 1 GB, which is 7.6 GB for the whole run with 4 workers. Our MapReduce programs only look at one line at a time: the mapper, combiner

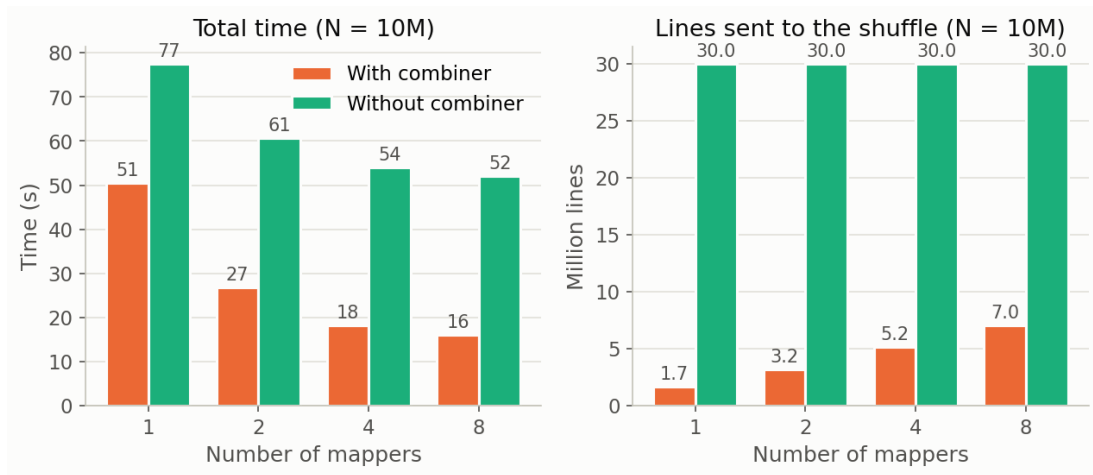


Figure 5: MapReduce with and without the combiner, 10M records.

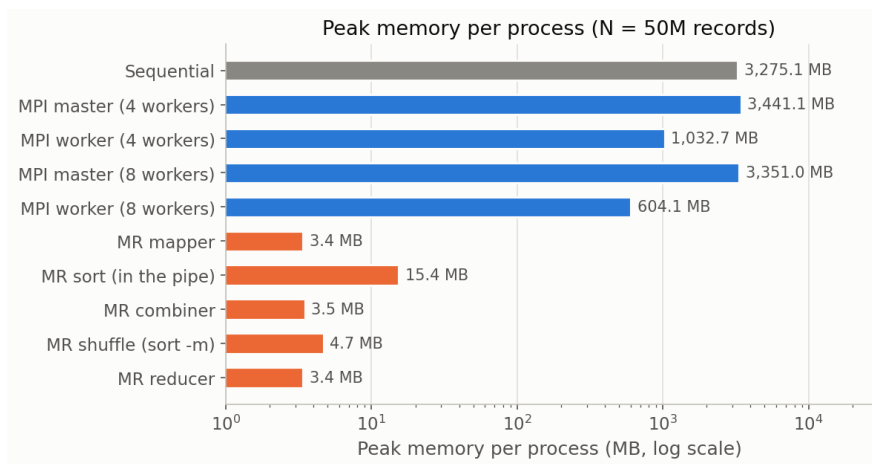


Figure 6: Peak memory of each process, 50M records (log scale).

and reducer each used about 3.5 MB, and `sort` used about 15 MB (it writes to temporary files on disk when needed). The whole MapReduce run used only 91 MB, about **80 times less**. This means MPI cannot handle data bigger than one machine’s memory, while MapReduce can.

Running on 4 nodes was slower (Fig. 4b). We also spread the 8 mappers over 4 nodes. This made the run slower (82 s $\rightarrow$ 109 s); only the map phase got slower. The reason is that **the nodes are not equally fast**. All our 1-node runs happened on node06, but the 4-node run also used node01–03. In the parallelism check, the same pieces took 11.9 s on node06 but 14.1–15.3 s on node01–03, and running all 8 pieces one after another took 45.9 s on node06 but 60.7 s on node01. The map phase can only finish when the *slowest* node finishes, so the slower nodes decide the time. (At first we thought reading the pieces over the shared network drive was the cause, but then node01, which wrote the pieces, would have been the fastest, and it was not.) Real Hadoop handles slow nodes by starting a backup copy of a slow task on another node (*speculative execution*), and it stores the input on the workers’ own disks (HDFS), so each map task can read its piece locally.

1.7 MPI vs MapReduce: summary

	MPI (HW2)	MapReduce
Time at 50M (best)	55.8 s	81.5 s
Speedup, $1 \rightarrow 8$ workers	none (rank 0 reads the input alone)	3.1 $\times$
Memory at 50M	7.6 GB (3.4 GB on rank 0)	91 MB
Data sent	binary structs	text lines, reduced by the combiner
Code we had to write	custom MPI datatype, scatter, 9 different send/receive messages, merging	3 small programs, no communication code
Adding a new statistic	change the struct, the messages and the merge	add one new key
If a process crashes	the whole job fails	Hadoop re-runs only that task

Conclusion. For our 2 GB input on one machine, MPI is faster because it works on binary data in memory. MapReduce is slower because of all the text conversion and sorting, but it gets faster with more mappers, uses 80 times less memory, and was much simpler to write. For data that does not fit on one machine, MapReduce is the better choice.

2 Q2: Real-time streaming analytics using gRPC

Here the same Q8 analytics run on a *stream*: records arrive one after another, and the analytics must be up to date at any moment while data is still coming in. As instructed by the TAs, Q2 is written in **Python with grpcio**. The code and run instructions are in `q2_grpc/README.md`.

2.1 Architecture

Program	Role
<code>client.py</code>	reads the dataset and streams it to the coordinator in batches, at a chosen rate
<code>coordinator.py</code>	gRPC server: splits every batch between the workers, answers queries by merging the workers' results
<code>worker.py</code> (W processes)	gRPC server: keeps a Stats object for the records it gets
<code>dashboard.py</code>	CLI dashboard: live view of the analytics; also <code>--once</code> , <code>--bench</code> , <code>--shutdown</code>
<code>weather_stats.py</code>	the Q8 analytics in Python (port of HW2's <code>q8_common.cpp</code>): update, merge, top- K , output

The client sends the header (N, K, S) with **Configure**, then the records over one **StreamRecords** call. The coordinator splits each batch into one small batch per worker and puts it into that worker's **queue**; one **sender thread per worker** sends the batches with **ProcessBatch**, so all workers get data at the same time. Each worker adds its records to its own **Stats** object, exactly like an HW2 MPI worker. When a dashboard calls **GetAnalytics**, the coordinator asks all workers for their partial **Stats** at the same time (**GetPartial**), merges them with the same merge rule as HW2, and returns the result in the HW2 output format together with live counters (records received and processed, ingest rate, records per worker). Because merging works at any moment, a query in the middle of the stream simply shows the analytics of the records processed so far.

2.2 The gRPC interface

Service	RPC	Why
Coordinator	Configure (StreamConfig) $\rightarrow$ Ack StreamRecords (stream RecordBatch) $\rightarrow$ Ack GetAnalytics $\rightarrow$ AnalyticsReply	sends N, K, S and resets the analytics, so the system can be reused client streaming : the whole dataset over one call; returns when all is processed current analytics (HW2 text) + live counters, any time during the stream
Worker	Configure , ProcessBatch (RecordBatch) GetPartial $\rightarrow$ PartialStats	a worker gets its share in batches the worker's partial Stats (same fields as HW2's struct)
Both	Shutdown	used by our scripts to stop the system

A `RecordBatch` holds many records, so the client decides how many records go into one message (the batch size).

2.3 Important decisions

- **Distribution strategy.** By default a record goes to worker $\lfloor t/60 \rfloor \bmod W$ (*by interval*), so every 60-second interval lives on exactly one worker. Then each worker knows the complete count of its intervals, and it only has to report its single busiest interval: the global busiest interval is the best of these. A query message stays small (the global fields, S station totals and one interval), however much data has arrived. For comparison we also implemented *round-robin* (records to the workers in turn): perfectly even load, but the records of one interval end up on several workers, so every query must send all interval counts of every worker. Top- K works with both: all $S = 100$ station totals are sent and added up before the top K are chosen.
- **One process per worker.** In Python only one thread per process can run Python code at a time (the GIL), so workers are separate processes; they then really compute in parallel. Threads are still used for waiting on the network.
- **Concurrency.** A worker's `ProcessBatch` and `GetPartial` can arrive at the same time (gRPC uses a thread pool), so a lock protects its `Stats`: a query always sees whole batches. The coordinator's queues have a maximum size, so a slow worker slows the stream down (*back pressure*) instead of filling the memory. `StreamRecords` returns only when all queues are empty, so the answer after the stream always includes every record.
- **Busiest interval kept up to date.** At first a worker found its busiest interval by scanning all its intervals on every query, while holding its lock. With dashboards querying non-stop this blocked the workers most of the time (4 dashboards: 45K records/s). Since counts only go up, it is enough to compare the one interval that just changed with the current best, so `update()` now keeps the busiest interval up to date and a query just reads it.
- **Cluster setup.** The cluster's default Python is 3.6, too old for `grpcio`, so we use Python 3.11 in a virtual environment with only `grpcio` and `protobuf` (the generated gRPC code is committed; plots are made on the laptop). The cluster also sets `http_proxy`, and Python gRPC would send even local connections through it, so all our channels switch the proxy off (`grpc.enable_http_proxy = 0`).

2.4 Correctness

`verify_correctness.sh` runs 253 checks, both on the laptop and on the RCE cluster (all passed on both): our Python analytics against HW2's C++ `sequential` on all 13 test cases; all 13 test cases through the gRPC system with 1, 2 and 4 workers, both strategies and batch sizes 1, 7 and 1000 (234 runs); 100K and 1M generated records; and 4 dashboards querying non-stop during a 1M-record stream, where no query failed, the processed count never went down and never exceeded N , and the final result was correct. Every benchmark run below is also checked against `sequential`.

Besides the rounding of very large sums (Q1), one more rounding effect appeared: a **rounding tie**. Every input value has 2 decimals, so the sum of the temperatures is a multiple of 0.01, and the average over $N = 20,000$ records is a multiple of 0.0000005. The exact average can therefore lie exactly halfway between two 6-decimal numbers (e.g. 14.9807675), and the tiny rounding error of the sum decides whether it is printed as 14.980767 or 14.980768. `compare_outputs.py` therefore also accepts a difference of one unit in the 6th decimal; counts, ids and the top- K order must still match exactly.

2.5 Demo on the RCE cluster

Following the RCE guide, `run_cluster.sh` (inside `salloc --nodes=4`) started the coordinator on node01 and 4 workers on node02, node03 and node06. The two dashboards ran on node03 and node06 and the client on node02, each in its own terminal, so every connection went over the cluster network. The client streamed 1M records at 100,000 records/s. While the stream was running, both dashboards showed the growing analytics (Fig. 7, left: 405,000 records after 4s, queries in about 10 ms). At the end they showed all 1,000,000 records (right), and the final analytics are exactly the HW2 `sequential` output for this dataset (e.g. `AVERAGE_TEMPERATURE 14.999518`, `TOTAL_RAINFALL 24999581.170000` and the same top-10 stations).

2.6 Experimental setup

The benchmarks ran on a laptop (Apple M4: 4 performance + 6 efficiency cores, 16 GB; Python 3.13, `grpcio` 1.84), with every program as a separate process on `localhost`; the TAs allowed running gRPC locally, and the

```

===== Weather analytics dashboard =====
coordinator      : node01:50110
status           : STREAMING
records processed : 405,000 (received 405,000)
elapsed          : 4.0 s
ingest rate      : 100,003 records/s
workers          : 4 (strategy interval)
records per worker: 101,707 101,228 101,158 100,907
query latency    : 10.40 ms

-----
TOTAL MEASUREMENTS 405000
AVERAGE_TEMPERATURE 15.028518
MIN_TEMPERATURE -20.000000
MAX_TEMPERATURE 50.000000
AVERAGE_HUMIDITY 49.930290
MIN_HUMIDITY 0.000000
MAX_HUMIDITY 100.000000
AVERAGE_PRESSURE 999.969351
MIN_PRESSURE 950.000000
MAX_PRESSURE 1050.000000
TOTAL_RAINFALL 10110829.630000
MAX_RAINFALL 50.000000
AVERAGE_WIND_SPEED 19.980880
MAX_WIND_SPEED 40.000000
EXTREME_TEMPERATURE_EVENTS 173766
HOTTEST_MEASUREMENT 50.000000 4 64917
COLDEST_MEASUREMENT -20.000000 7 465200
BUSIEST_INTERVAL 71779 12
TOP_STATIONS
4 4224 14.843222 105788.480000
44 4218 14.879870 106866.560000
92 4179 15.264422 104903.780000
50 4169 15.444248 105431.260000
64 4167 14.337905 102860.650000
18 4162 15.415147 104984.620000
8 4152 15.046344 102456.370000
73 4147 15.044533 103324.080000
81 4143 15.320623 104718.320000
27 4139 15.184791 104117.270000

```

(a) During the stream (node03).

```

===== Weather analytics dashboard =====
coordinator      : node01:50110
status           : DONE
records processed : 1,000,000 (received 1,000,000)
elapsed          : 10.0 s
ingest rate      : 99,986 records/s
workers          : 4 (strategy interval)
records per worker: 250,420 249,573 250,236 249,771
query latency    : 6.24 ms

-----
TOTAL MEASUREMENTS 1000000
AVERAGE_TEMPERATURE 14.999518
MIN_TEMPERATURE -20.000000
MAX_TEMPERATURE 50.000000
AVERAGE_HUMIDITY 49.997032
MIN_HUMIDITY 0.000000
MAX_HUMIDITY 100.000000
AVERAGE_PRESSURE 999.977290
MIN_PRESSURE 950.000000
MAX_PRESSURE 1050.000000
TOTAL_RAINFALL 24999581.170000
MAX_RAINFALL 50.000000
AVERAGE_WIND_SPEED 19.982403
MAX_WIND_SPEED 40.000000
EXTREME_TEMPERATURE_EVENTS 428753
HOTTEST_MEASUREMENT 50.000000 19 27739
COLDEST_MEASUREMENT -20.000000 14 94133
BUSIEST_INTERVAL 8336 20
TOP_STATIONS
4 10296 15.014771 258070.100000
73 10251 15.116503 254639.260000
79 10219 15.050550 257887.590000
86 10180 15.013839 255007.160000
8 10175 15.177221 253216.510000
23 10155 14.969888 253972.790000
80 10153 14.954394 254784.320000
28 10135 15.050652 251036.480000
81 10121 15.185716 253924.400000
66 10107 15.387017 251664.200000

```

(b) After the stream.

Figure 7: The CLI dashboard during the cluster demo (coordinator on node01, 4 workers, client on node02).

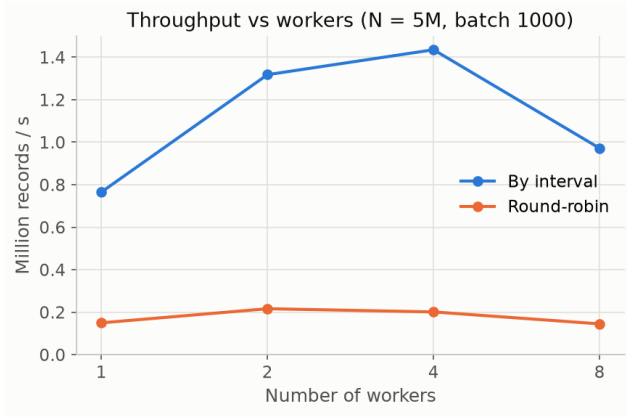
cluster queue was full close to the deadline. Correctness (253/253) and the multi-node demo (Section 2.5) were done on the RCE cluster. Datasets come from the HW2 generator (seed 42, $K = 10$, $S = 100$): 5M records for most runs, 200K for the batch-size runs. Every run starts a fresh system, and its final result is checked against HW2’s `sequential`: all 22 runs were correct (5 EXACT, 17 FP_CLOSE). The client builds all messages before it starts, and the time is measured from the first record sent until the coordinator confirms that *all* records are processed (end to end). On a laptop, repeated runs can differ by up to about 30%. The generator draws timestamps uniformly from $[0, 10N]$ (about 6 records per 60-second interval) and the values uniformly from fixed ranges with 2 decimals (details and the exact commands are in the README).

Why these configurations. 5M records make every run last several seconds (3.5–35 s), so start-up costs and noise matter less, while all 22 runs still fit in about 15 minutes. Batch size 1000 is used everywhere else because the batch-size experiment shows it already reaches most of the maximum speed. 1 to 8 workers covers the laptop’s 10 cores (with 8 workers the processes already outnumber the cores). The workers experiment uses one dashboard refreshing every 100 ms, like a real live view; the queries experiment uses dashboards that query back to back, the worst case. The requested rates go from well below to just above the measured maximum.

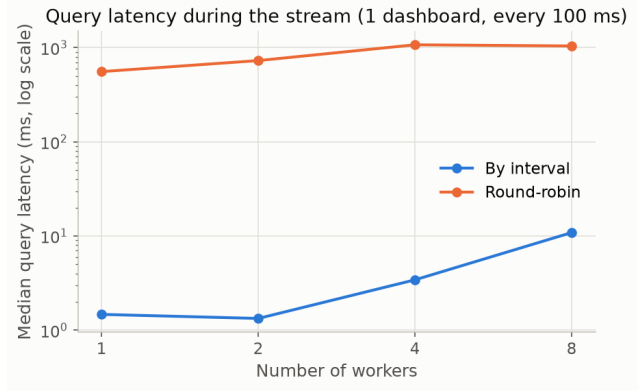
2.7 Results

More workers help, up to the coordinator’s limit (Fig. 8a). With the interval strategy the throughput goes from 0.77M records/s (1 worker) to 1.32M (2) and 1.44M (4). Unlike in C++, the analytics in Python is real work: one worker alone manages about 0.8M records/s, so a second worker almost doubles the speed. After that the **coordinator** becomes the limit: it is a single Python process that touches every record to decide which worker gets it. With 8 workers it drops to 0.97M: 8 workers, the coordinator, the client and the dashboard are 11 busy processes on 10 cores (6 of them slower efficiency cores), so they slow each other down.

The distribution strategy matters a lot (Fig. 8b). With round-robin, the *same* runs reach only 0.15–0.22M records/s, and a query takes 0.56–1.07 s instead of 1.3–11 ms. The reason is the busiest interval: with round-robin every worker has a part of almost every interval (about 830K intervals at 5M records), so for every query each worker must put all its interval counts into the reply, while holding its lock. That takes most of a second, and a dashboard asks every 100 ms, so the workers are blocked most of the time and ingestion collapses. With the interval strategy a worker sends only its busiest interval, so a reply is tiny whatever the amount of data. Round-robin has perfectly even load (exactly 1.25M records per worker at $W = 4$), but the interval



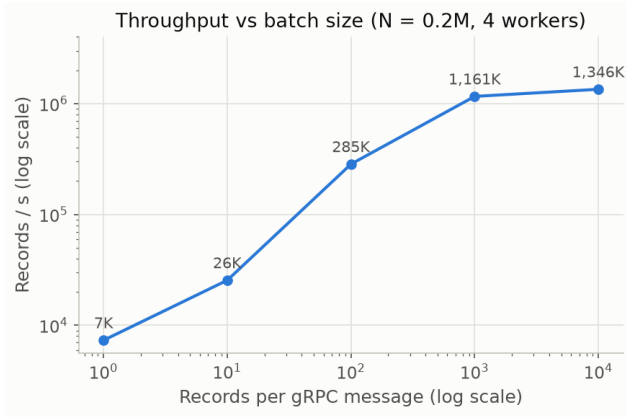
(a) Throughput vs workers.



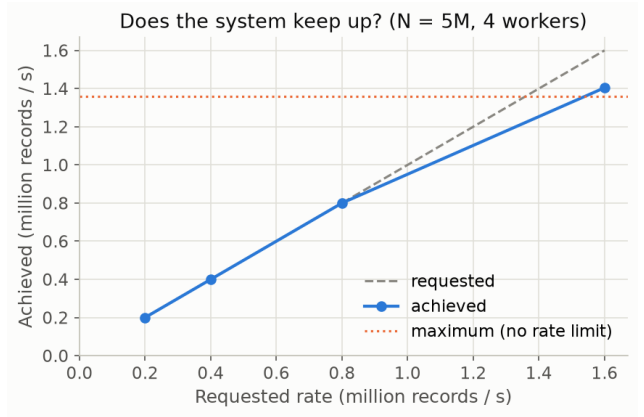
(b) Query latency vs workers.

Figure 8: Number of workers and distribution strategy (5M records, batch 1000, one dashboard querying every 100 ms).

strategy is almost as even (1.248M–1.252M) because timestamps are spread uniformly. Query latency grows a little with W for the interval strategy (more workers to ask and more partial results to merge, and more processes sharing the cores).



(a) Throughput vs batch size (200K records).



(b) Requested vs achieved stream rate (5M).

Figure 9: How the records are streamed (4 workers, interval strategy).

Batch size: every message has a fixed cost (Fig. 9a). With 1 record per message the system manages only 7.3K records/s; with 10 records 25.5K, 100 records 285K, 1000 records 1.16M and 10000 records 1.35M. Every message costs the same fixed amount of work, whatever its size: gRPC framing, a Python call on both sides, and a hand-over between threads, once from the client to the coordinator and once more from the coordinator to each worker. With 1 record per message that is about 130 μ s per record, so 200K records take 27s. With 1000 or more records per message the fixed cost is shared by many records and the per-record work (reading and updating the statistics) dominates, so bigger batches help less and less.

Stream rate: the system keeps up to about 1.4M records/s (Fig. 9b). When the client sends at a fixed rate, the system processes exactly that rate: 200K, 400K and 800K records/s requested gave 199,999, 399,989 and 799,951 records/s. At 1.6M requested it reaches only 1.40M, the same as without any limit (1.36M): that is the maximum speed of this setup. If the stream is faster than that, the coordinator’s full queues make the stream wait (back pressure), so nothing is lost and the memory does not grow; the data just arrives later.

Queries during the stream cost ingestion speed (Fig. 10). With 0, 1, 4 and 16 dashboards querying back to back, the throughput goes from 1.44M to 1.09M, 0.77M and 0.58M records/s, and the median query latency from 3.3 to 6.0 and 19ms. The queries still get fast answers (16 dashboards together answered about 7,200 queries in 12s), but every query takes each worker’s lock for a moment and runs in the coordinator’s Python process, which also routes the stream, so the two compete. A normal dashboard that refreshes every

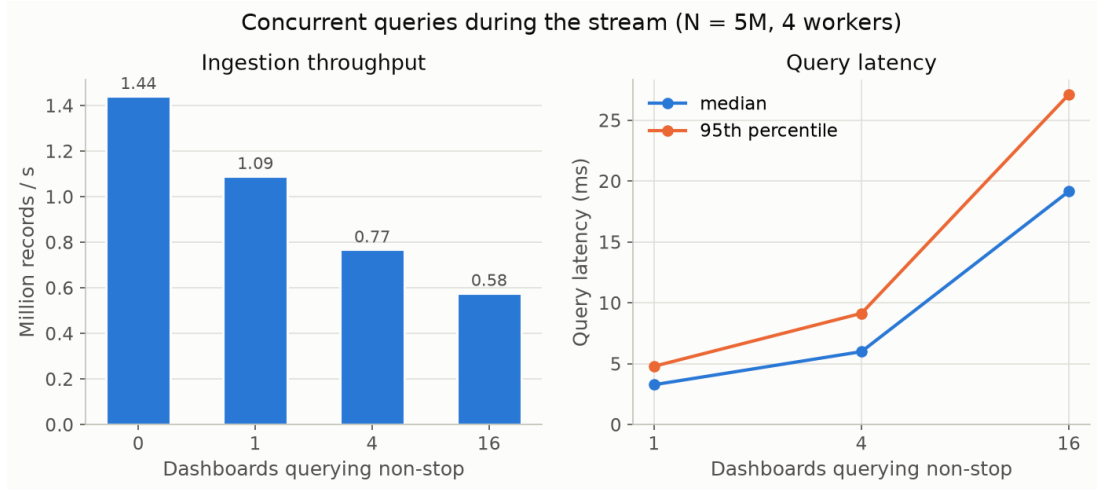


Figure 10: Dashboards querying non-stop during the stream (5M records, 4 workers, interval strategy).

few hundred milliseconds costs almost nothing; only many dashboards querying non-stop slow the stream down noticeably. (This is after the fix that keeps the busiest interval up to date; before it, 4 non-stop dashboards brought the stream down to 45K records/s.)

Memory and CPU. We measured every process of one run (5M records, 4 workers, one dashboard every 100 ms) with `measure_resources.sh`:

Process	By interval		Round-robin	
	peak memory	CPU time	peak memory	CPU time
coordinator	66 MB	5.8 s	646 MB	21.1 s
each worker	60 MB	2.6 s	181–195 MB	7.0 s
client (5M records prepared as messages)	563 MB	5.7 s	563 MB	5.9 s
dashboard	33 MB	0.3 s	34 MB	0.3 s

Workers keep only statistics, never the records, so with the interval strategy they need about 60 MB (mostly Python itself plus the interval counts) no matter how long the stream is. Round-robin needs 10× more memory in the coordinator and 3–4× more CPU in total for the same data: every query copies all interval counts of every worker into messages, sends them, and merges them. The coordinator uses more CPU time than the run lasts (5.8s in 3.6s) because gRPC does its network work in its own threads, next to the Python thread that routes the records.

2.8 Streaming (gRPC) vs batch (MapReduce, MPI)

	MPI (HW2, C++)	MapReduce (Q1, C++)	gRPC streaming (Q2, Python)
Data	complete file, in memory	complete file, on disk	records arrive continuously
When is the answer ready	at the end	at the end	any time (queries take milliseconds)
What limits the speed	rank 0 reading the input alone	text conversion, sorting, serial split	the single coordinator routing every record, per-message cost
Memory	all records in memory	a few MB per process	only the statistics (no records are kept)

Their speeds are not directly comparable (different languages and machines), but the models are. MPI and MapReduce answer once, after all data is there. The streaming system gives an answer at any moment, in milliseconds, because every worker keeps its statistics up to date and a query only has to merge the small partial results. The idea is the same in all three: partial results that can be merged. What changes is when the merge happens (once at the end, or at every query) and what has to be sent for it; the second point is exactly why the interval strategy is so much better than round-robin here.